

Trabajo Practico Integrador de POO2

Informe

Integrantes del grupo:

- Facundo Lezcano
- Germán Scheffelaar Klotz
- Frank Matias Flores Rea

Módulos

Módulo 2.1: Catálogo de Productos

Problema a resolver

El catálogo debe manejar tanto productos individuales como paquetes (que agrupan múltiples productos u otros paquetes). El sistema requiere calcular recursivamente el precio y los descuentos en toda la jerarquía, y el cliente que interactúa con el catálogo debe poder tratar a un producto y a un paquete de la misma manera, sin preocuparse por de qué tipo de elemento se trata. Además, los productos individuales deben poder incorporar atributos dinámicos.

Alternativa pobre de diseño

Una forma ineficiente de resolver el manejo de paquetes hubiese sido utilizar una única clase Producto con una lista interna de "sub-productos" y un atributo booleano (ej. esPaquete).

Esta alternativa viola el Principio Abierto/Cerrado (OCP), ensucia el código con condicionales evitables y hace que el cálculo recursivo de precios con descuentos anidados sea extremadamente frágil y propenso a errores.

Patrón utilizado para resolución de Módulo: Composite

Para representar esta jerarquía de "parte-todo" se optó por implementar el patrón **Composite**. Este patrón le permite al cliente tratar a los objetos individuales y a las composiciones de manera completamente uniforme mediante el polimorfismo.

Participantes del patrón (Roles GoF):

- **Component - Item:** Es la clase abstracta que declara la interfaz común para todos los elementos del catálogo.
- **Leaf (Hoja) - Producto:** Representa a los objetos individuales, que no tienen hijos. Su precio base se obtiene directamente de su atributo interno.
- **Composite - Paquete:** Representa un elemento complejo que contiene hijos. Su precio base y su peso se calculan delegando la operación a sus hijos y sumando los resultados.

Decisiones de diseño e implementación destacadas

- **Seguridad en el Componente:** Se decidió declarar los métodos de gestión de hijos (add() y remove()) directamente en la clase abstracta Item, los cuales lanzan un error por defecto. Solo el Paquete sobrescribe estos métodos para

darles comportamiento real, evitando que se agreguen ítems por error a un Producto.

- **Cálculo recursivo polimórfico:** La aplicación de descuentos sobre paquetes anidados se resuelve de manera natural y sin condicionales, dejando que la recursión del árbol y el polimorfismo resuelvan la matemática de los descuentos en cascada.
- **Aplanamiento de la estructura (getRegistroDelItem):** Se diseñó un mecanismo para que, al momento de la venta, el paquete pueda "desglosarse" recursivamente en una lista plana de RegistroDelItem. Esto permite que el sistema de reportes y ventas procese fácilmente las unidades individuales, calculando el "multiplicador de descuento" efectivo que le llegó a cada hoja del árbol.
- **Atributos dinámicos tipados:** Para resolver la escalabilidad de los atributos del Producto, se implementó un modelo de atributos dinámicos utilizando genéricos (Atributo<T>) y subclases específicas (AtributoString, AtributoNumero, AtributoBooleano). Esto evita el uso de un simple diccionario de tipo Map<String, Object>, previniendo errores y asegurando la consistencia de los tipos de datos.

Módulo 2.2: Ciclo de Vida del Pedido

Problema a resolver

El dominio establece que un pedido modifica su comportamiento a lo largo de su ciclo de vida y que, según su contexto actual, existen operaciones que son válidas y otras que no.

Alternativa pobre de diseño

Una forma poco conveniente de resolver este problema hubiese sido agregar un atributo de tipo Enum o un String dentro de la clase Pedido, y utilizar bloques case o múltiples if para verificar si el estado actual permite la operación.

Entre otras cosas, esta alternativa genera menor escalabilidad (agregar un nuevo estado requiere muchas modificaciones en el código) y es más propensa a errores.

Patrón utilizado para resolución de Módulo: State (Estado)

Para evitar la fragilidad mencionada, se optó por implementar el patrón **State**. Este patrón permite que un objeto altere su comportamiento cuando su estado interno cambia, pareciendo que el objeto cambia de clase.

Gracias a este patrón, polimorfizamos el comportamiento del pedido. Ahora, la clase Pedido delega la ejecución de sus operaciones a un objeto estado que representa su contexto actual.

Participantes del patrón (Roles GoF)

- **Context** - Pedido: Es la clase principal con la que interactúa el cliente. Mantiene una referencia a una instancia de EstadoPedido (el estado actual) y le delega todas las operaciones del ciclo de vida.
- **State** - EstadoPedido: Clase abstracta que define la interfaz común para todos los estados posibles.
- **Concrete States** - EstadoBorrador, EstadoConfirmado, EstadoEnPreparacion, etc.: Cada una de estas clases encapsula el comportamiento específico asociado a un estado particular del pedido e implementa las transiciones válidas dictadas por el dominio.

Decisiones de diseño e implementación

1. **Comportamiento por defecto de los estados:** En lugar de implementar EstadoPedido como una interfaz, se decidió implementarla como una clase abstracta donde **todos** los métodos por defecto lanzan la excepción de dominio OperacionInvalidaParaEstadoException. Así, los estados concretos solo sobrescriben los métodos de las operaciones que *sí* son válidas en su contexto.
2. **Reutilización de lógica con EstadoReembolsador:** Para evitar duplicar parte del código de cancelación, se introdujo una clase intermedia abstracta llamada EstadoReembolsador (que hereda de EstadoPedido). Esta clase aplica internamente una variante del patrón *Template Method*, definiendo : define la lógica de cancelar y generar la nota, pero delegando a sus subclases (EstadoEnPreparacion y EstadoEnviado) el cálculo de los "extras a reembolsar".
3. **Separación de responsabilidades en el Contenido:** Se decidió extraer toda la manipulación de la lista de ítems a la clase ContenidoPedido para evitar sobrecargar la clase principal y respetar el Principio de Responsabilidad Única (SRP).
4. **Integración transparente con el Patrón Observer:** Para desacoplar este módulo de subsistemas que deben ser notificados durante el ciclo de vida de un pedido, cada vez que un estado concreto ejecuta una transición exitosa, notifica automáticamente a los observadores suscritos.

Módulo 2.3: Metodos de Envio

Problema para resolver

El sistema debe calcular el costo y el tiempo de entrega de un pedido, contemplado distintas modalidades de envío(estándar, express y retiro en sucursal).Cada modalidad delega su cálculo en un proveedor externo distinto, con su propia interfaz

y el pedido debe poder utilizar cualquiera de ellas sin conocer los detalles particulares de cada proveedor.

Patrón utilizado para resolución de Módulo: Strategy

Se implementó el patrón Strategy para separar los algoritmos de cada tipo de envío. Esto permite tratar cada modalidad de envío de forma independiente. Pedido se limita a delegar la solicitud a la modalidad asignada, manteniendo la abstracción frente a las particularidades de cada uno.

Participantes del patrón (Roles GoF)

- **Context (Pedido):** Almacena la referencia a la estrategia activa y solicita los cálculos (días de Entrega y costo) y es posible alterar la estrategia dinámicamente mediante métodos de un método de asignación.
- **Strategy (MetodoDeEnvio):** Interfaz que estandariza las operaciones necesarias
- **Concrete Strategy (EnvioEstandar, EnvioExpress, RetiroASucursal):** Clases que encapsulan la lógica específica de cada modalidad.

Decisiones de diseño e implementación destacadas

- **Flexibilidad:** El cambio de modalidad es transparente y ocurre en tiempo de ejecución, eliminando estructuras condicionales.
- **Encapsulamiento:** Cada estrategia gestiona sus propias dependencias externas (ej. CorreoArgentino), ocultando esta complejidad al resto del sistema.
- **Conversión de datos:** Las estrategias se encargan de adaptar formatos (por ejemplo, diferencias de precisión entre float y double), aislando estas inconsistencias.
- **Extensibilidad:** Cumple estrictamente con el principio OCP; agregar una nueva forma de envío solo requiere crear una nueva clase concreta, sin tocar el código fuente existente de la clase Pedido

Módulo 2.4: Métodos de Pago

Problema para resolver

El procesamiento de un pago sigue siempre la misma secuencia de pasos: validar los datos ingresados, reservar los fondos necesarios, ejecutar la transacción y notificar el resultado. Sin embargo, cada medio de pago implementa esos pasos de manera diferente según sus propias reglas de negocio.

El sistema debe permitir incorporar distintos medios de pago manteniendo un flujo de procesamiento común, evitando la duplicación de lógica y garantizando que todas las transacciones respeten el mismo orden de ejecución.

Alternativa pobre de diseño

Una forma poco conveniente de resolver este problema hubiese sido implementar toda la lógica de procesamiento dentro de una única clase utilizando múltiples bloques if o switch para distinguir entre tarjeta de crédito, transferencia bancaria y billetera virtual.

Esta alternativa genera código difícil de mantener, duplica comportamiento común y obliga a modificar la lógica principal cada vez que se incorpora un nuevo medio de pago. Además, aumenta el riesgo de romper la secuencia de procesamiento establecida por el Template Method.

Patrón utilizado para resolución de Módulo: Template Method

Para resolver este problema se optó por implementar el patrón Template Method.

Este patrón permite definir el esqueleto general de un algoritmo en una clase base, delegando a las subclasses la implementación de los pasos específicos.

De esta manera, la clase MedioPago garantiza que todas las transacciones sigan la misma secuencia de procesamiento, mientras que cada medio de pago concreto implementa sus propias reglas de validación, reserva de fondos, ejecución y notificación.

Participantes del patrón (Roles GoF)

- **Abstract Class - MedioPago:** Define el algoritmo general de procesamiento mediante el método procesarPago() y declara los pasos abstractos que deben implementar las subclasses. Además, proporciona una implementación por defecto para la generación del código de transacción.
- **Concrete Class - TarjetaCredito:** Implementa la validación de los datos de la tarjeta, la preautorización de fondos y la ejecución de la transacción mediante la API correspondiente. Personaliza la notificación generando un Cupón de Pago.
- **Concrete Class - TransferenciaBancaria:** Implementa la validación de la cuenta bancaria y la ejecución de la transferencia. Personaliza la notificación generando un Comprobante de Transferencia.
- **Concrete Class - BilleteraVirtual:** Implementa la validación de saldo disponible, el bloqueo de fondos y la acreditación al vendedor. Personaliza la notificación mediante el envío de una notificación push al usuario.

Decisiones de diseño e implementación destacadas

- **Centralización del procesamiento de pagos:** La clase MedioPago concentra el algoritmo completo mediante el método procesarPago(), garantizando que todos los medios de pago respeten la misma secuencia de validación, reserva, ejecución y notificación definida por el dominio.
- **Generación centralizada de códigos de transacción mediante Hook Method:** El método notificarResultado() proporciona una implementación por defecto que genera y registra un código de transacción único para todos los pagos procesados. Las subclasses extienden este comportamiento para incorporar acciones específicas, como la generación de comprobantes o el envío de notificaciones, sin modificar el Template Method.
- **Comprobantes y notificaciones específicas por medio de pago:** Cada medio de pago puede producir distintas evidencias de una transacción exitosa. Para ello se modelaron objetos específicos como CuponPago y ComprobanteTransferencia, mientras que BilleteraVirtual personaliza la notificación mediante mensajes push al usuario.
- **Integración desacoplada con servicios externos:** Las validaciones y operaciones específicas de cada medio de pago se delegan a interfaces independientes (APITarjetaCredito, APITransferenciaBancaria y APIBilleteraVirtual), evitando acoplar el dominio a implementaciones concretas.

Módulo 2.5: Notificaciones del Pedido

Problema a resolver

Cada vez que un pedido transiciona de estado, múltiples subsistemas necesitan reaccionar a ese evento: enviar un email al cliente, generar una factura, otorgar un cupón de fidelización o registrar la venta en el historial. El sistema debe permitir que estos subsistemas se enteren de los cambios de estado sin que el Pedido necesiten conocerlos explícitamente ni modificarse cada vez que se agrega un nuevo interesado en ser notificado.

Alternativa pobre de diseño

Como alternativa al diseño de notificaciones, se podría haber intentado representar los estados mediante un Enum o utilizar verificaciones explícitas como instanceof o getClass() dentro de los observadores para determinar qué acciones ejecutar. Estas estrategias son altamente ineficientes, ya que obligan a los observadores a contener lógica condicional rígida para filtrar los estados de su interés, violan el principio de encapsulamiento al requerir conocimiento de los tipos concretos de estado, y rompen el Principio Abierto/Cerrado al exigir modificaciones en múltiples clases cada vez que se agrega un nuevo evento al ciclo de vida del pedido.

Patrón utilizado para resolución de Módulo: Observer

Para resolver este problema se optó por implementar el patrón Observer. Este patrón permite definir una dependencia de uno a muchos entre el Pedido y los distintos subsistemas interesados, de modo que, cuando el pedido cambia de estado, todos sus observadores suscritos sean notificados automáticamente sin que el Pedido conozca sus implementaciones concretas.

Participantes del patrón (Roles GoF)

- **Subject(Pedido):** Mantiene conjunto de observadores suscritos y luego delega cada transición en su estado actual que dispara la notificación mediante `notificarTransicion()`.
- **Observer(ObserverPedido):** Clase abstracta que declara los métodos `hook alConfirmar()`, `alEnviar()`, `alCancelar()`, `alEntregar()`, todos con una implementación vacía por defecto, de modo que cada observador que herede de ella solo implemente los métodos que le interesan.
- **Concrete Observer(NotificadorEmail - Fidelización - GeneradorDeFactura):** Cada uno reacciona de forma distinta antes los distintos eventos de pedido, el `NotificadorEmail` envía correos a los clientes, `fidelizacion` genera un cupon y envía un mail con el cupón de descuento del 5% y el `GeneradorDeFactura` genera un comprobante fiscal.

Decisiones de diseño e implementación destacadas

- **Independencia del módulo de pedido:** tanto `Pedido` como `EstadoPedido` desconocen las clases concretas de los observadores, solo conocen a la abstracción `ObservadorPedido` permitiendo incorporar nuevos observadores sin modificar el ciclo de vida del pedido.
- **Integración desacoplada con servicios externos:** el envío de correos y la generación de comprobantes se delegan en las interfaces `MailSender` y `ComprobanteFiscal` respectivamente, evitando acoplar el módulo de notificaciones a una implementación concreta de estos servicios.
- **Doble Dispatch delegado en el State:** en lugar de que el propio `Pedido` decida, mediante condicionales, qué método del observador corresponde invocar según el estado alcanzado, esa decisión se delega en el propio `EstadoPedido` a través de `notificarTransicion(pedido, observador)`. Cada estado concreto sabe a qué método del observador debe llamar (por ejemplo, `EstadoConfirmado` invoca `observador.alConfirmar(pedido)`), logrando así una integración completamente polimórfica entre los patrones `State` y `Observer`, sin `instanceof` ni `switch` sobre tipos.
- **Puente hacia el módulo de ventas:** `ObservadorHistorialDeVentas` es el único punto de conexión entre este módulo y el módulo de reportes: al ser notificado de la entrega del pedido, traduce ese evento de dominio en un registro de venta dentro de `HistorialDeVentas`.

Módulo 2.6: Búsqueda en el Catálogo

Problema a resolver

El sistema debe permitir que los clientes encuentren productos y paquetes dentro del catálogo aplicando distintos criterios de búsqueda. Estos criterios pueden utilizarse individualmente o combinarse entre sí para construir consultas más complejas. La búsqueda debe operar sobre la colección completa de ítems del catálogo y devolver únicamente aquellos elementos que satisfacen la condición solicitada. Además, el catálogo no debe tomar decisiones sobre cómo evaluar los ítems, sino delegar esa responsabilidad a los criterios de búsqueda utilizados.

Alternativa pobre de diseño

Una forma poco conveniente de resolver este problema hubiese sido implementar toda la lógica de búsqueda dentro de la clase Catalogo mediante múltiples bloques if o switch que evaluaran cada posible criterio y sus distintas combinaciones. Esta alternativa genera acoplamiento entre el catálogo y las reglas de búsqueda, dificulta la incorporación de nuevos criterios y obliga a modificar la clase principal cada vez que aparece una nueva condición o combinación lógica.

Además, la construcción de consultas complejas mediante operadores como AND, OR y NOT se vuelve difícil de mantener, propensa a errores y obliga a extender continuamente la lógica del catálogo.

Patrón utilizado para resolución de Módulo: Composite

Para resolver este problema se implementó el patrón de diseño Composite, representando cada criterio de búsqueda como un objeto independiente que implementa la interfaz CriterioBusqueda.

Los criterios simples encapsulan una única condición de búsqueda y evalúan directamente cada ítem del catálogo. Por su parte, los criterios compuestos (And, Or y Not) contienen otros criterios de búsqueda, delegan en ellos la evaluación y combinan sus resultados para construir consultas de cualquier nivel de complejidad. Gracias a que todos implementan la misma interfaz, el catálogo puede trabajar de manera uniforme con cualquier criterio de búsqueda, sin distinguir si se trata de un criterio simple o compuesto, manteniendo desacoplada toda la lógica de búsqueda.

Participantes del patrón (Roles GoF)

- **Component – CriterioBusqueda:** Define la interfaz común que implementan todos los criterios de búsqueda mediante el método cumple(Item item). Gracias a esta abstracción, tanto los criterios simples como los compuestos pueden tratarse de manera uniforme.

- **Leaf (Hoja) -PorNombre, PorCategoria, PorPrecioMaximo, PorDisponibilidad:** Representan los criterios simples de búsqueda. Cada uno implementa directamente la evaluación de un Ítem según una condición específica y no contiene otros criterios de búsqueda.
- **Composite - And, Or, Not:** Representan los criterios compuestos. Mantienen referencias a otros objetos de tipo CriterioBusqueda, delegan la evaluación en ellos mediante el método cumple(Item) y combinan los resultados utilizando los operadores lógicos correspondientes.
- **Client – Catálogo:** Recorre la colección de ítems del catálogo y delega la evaluación al criterio recibido. El catálogo trabaja únicamente con la interfaz CriterioBusqueda, sin conocer si el criterio utilizado es simple o compuesto.

Decisiones de diseño e implementación destacadas

- **Desacoplamiento de responsabilidades:** La clase Catálogo se limita a recorrer los ítems disponibles y delegar su evaluación al criterio recibido mediante el método cumple(Item). De esta manera, el catálogo no necesita conocer las reglas específicas de búsqueda ni distinguir entre los distintos criterios implementados.
- **Composición de criterios de búsqueda:** Los operadores And, Or y Not fueron implementados como criterios de búsqueda. Esto permite combinarlos recursivamente para construir consultas simples o complejas sin modificar el catálogo ni los criterios existentes.
- **Integración desacoplada con Inventario:** La validación de disponibilidad se resolvió mediante la clase ConsultaDisponibilidadInventario, que actúa como intermediaria entre el módulo de búsqueda y el inventario, evitando dependencias directas entre ambos subsistemas.
- **Reutilización del Composite del módulo Catálogo:** El criterio PorNombre reutiliza el método coincideNombre() definido en la jerarquía Ítem. Gracias a ello, un Producto evalúa su propio nombre, mientras que un Paquete también verifica el nombre de cualquiera de los ítems que contiene, reutilizando el comportamiento recursivo implementado en el módulo Catálogo sin duplicar lógica de búsqueda.

Módulo 2.8: Reportes

Problema a resolver

El sistema debe poder generar reportes sobre el estado del negocio (como los productos más vendidos) y permitir exportarlos en múltiples formatos de salida (texto plano, CSV y HTML). Es un requisito crítico que el núcleo lógico de cada

reporte —es decir, cómo se recolectan, calculan y estructuran los datos— sea completamente independiente del formato visual en el que se presente.

Alternativa pobre de diseño

Una forma frágil de abordar este problema sería incluir un método exportar(Formato formato) dentro de la clase del reporte, utilizando sentencias bloques case o condicionales para definir cómo ordenar los datos según el formato solicitado. Esta alternativa viola el Principio de Responsabilidad Única (SRP) al mezclar lógica de negocio con lógica de presentación. Además, viola el Principio Abierto/Cerrado (OCP), ya que agregar un nuevo formato de exportación llevaría irremediablemente a grandes cambios de código.

Patrón utilizado para resolución de Módulo: Visitor (Visitante)

Para resolver este problema se optó por implementar el patrón **Visitor**. Este patrón permite separar la lógica estructural y matemática de recolección de datos de las operaciones que se realizan sobre esos datos (en este caso, formatearlos). Además, le permite al sistema agregar nuevos formatos de exportación en el futuro sin modificar la clase del reporte.

Participantes del patrón (Roles GoF)

- **Visitor - FormatoVisitante:** Interfaz que declara el contrato para los visitantes mediante el método visitar(ReporteDeProductosMasVendidos).
- **Concrete Visitor - FormatoTXT, FormatoCSV, FormatoHTML:** Clases que implementan la lógica de presentación específica para cada formato. Extraen la información procesada por el reporte y construyen el String final.
- **Element - Reporte:** Interfaz que define el método de entrada aceptar(FormatoVisitante).
- **Concrete Element - ReporteDeProductosMasVendidos:** Clase que encapsula la lógica de negocio real (buscar en el historial de ventas, calcular promedios y ordenar por cantidad). Implementa el método aceptar con un Double Dispatch.

Decisiones de diseño e implementación destacadas

- **Desacoplamiento estricto (SRP y OCP):** Se buscó la buena división de responsabilidades. La clase ReporteDeProductosMasVendidos se encarga de procesar la información de las ventas, pero ignora el cómo se mostrarán esos datos. Por su parte, los formatos se encargan de la presentación de los datos, pero no tienen responsabilidad matemática alguna.
- **Registro e historial de ventas:** Se decidió crear un ecosistema propio para el registro histórico separado del ciclo de vida del pedido. La Venta y el HistorialDeVerntas actúa como un registro inmutable que facilita filtrar a las

ventas registradas por rango de fechas sin acoplar los reportes a los pedidos en curso.

- **Uso de una estructura intermedia (LineaDeReporte):** Para facilitar el trabajo de los visitantes y no exponer la lógica de cálculo interno, el reporte consolida la información en una lista de objetos LineaDeReporte. Esta clase actúa como una estructura poseedora de los datos ya procesados (nombre del ítem, cantidad vendida y precio promedio) que los visitantes pueden iterar y formatear fácilmente.
- **Reutilización del comportamiento en cascada (getRegistroDelItem):** Para poder calcular las cantidades vendidas, el reporte se apoya indirectamente en el composito del módulo Catálogo. Al invocar a `getRegistroDelItem()`, logra que los paquetes anidados se aplanen en productos individuales reales, facilitando el análisis de los datos sin importar la complejidad original de la compra.